QUESTION:9

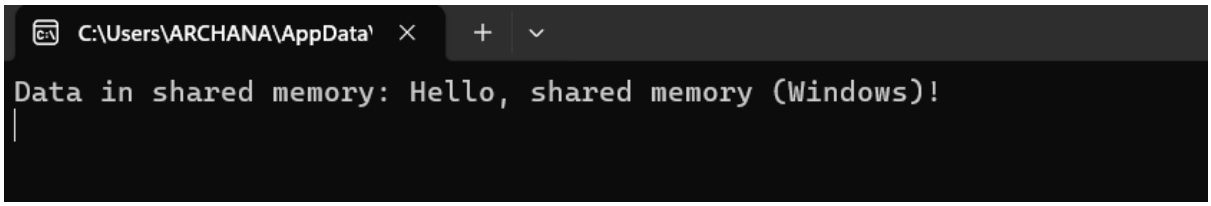
Illustrate the concept of inter-process communication using shared memory with a C program.

PROGRAM:

```
#include <windows.h>
#include <stdio.h>
#include <string.h>
int main(void) {
   const DWORD SIZE = 1024;
   const char *name = "Local\\MyShmExample";
   const char *msg  = "Hello, shared memory (Windows)!";
   // Create or open a named shared memory mapping
   HANDLE hMap = CreateFileMappingA(INVALID_HANDLE_VALUE, NULL,
                                    PAGE_READWRITE, 0, SIZE, name);
   if (!hMap) {
      printf("CreateFileMapping failed (%lu)\n", GetLastError());
      return 1;
   }
   // Map it into our address space
   char *p = (char*)MapViewOfFile(hMap, FILE_MAP_ALL_ACCESS, 0, 0, 0);
   if (!p) {
      printf("MapViewOfFile failed (%lu)\n", GetLastError());
      CloseHandle(hMap);
      return 1;
   }
   // Write then read
   strcpy(p, msg);
   printf("Data in shared memory: %s\n", p);
   UnmapViewOfFile(p);
   CloseHandle(hMap);
   getchar(); // keep console open (optional)
```

```
    return 0;
}
```

OUTPUT:



QUESTION:10

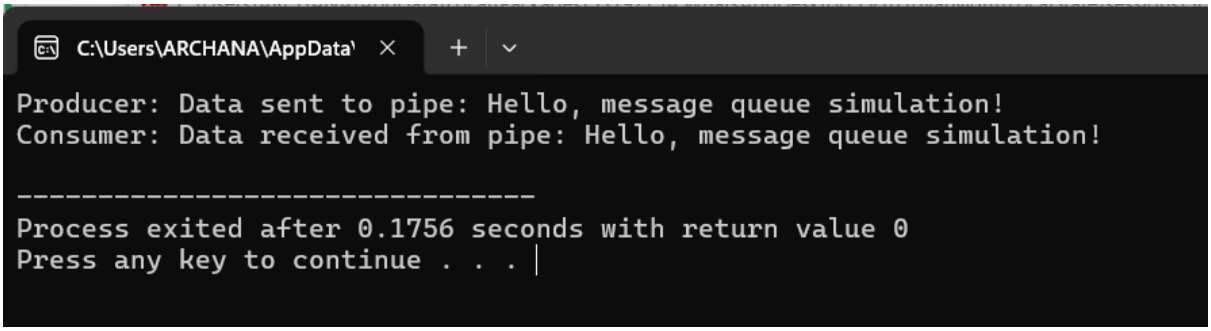
Illustrate the concept of inter-process communication using message queue with a c program.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
int main() {
    HANDLE hRead, hWrite;
    char buffer[100];
    DWORD bytesRead, bytesWritten
    // Create a pipe
    if (!CreatePipe(&hRead, &hWrite, NULL, 0)) {
        printf("Error creating pipe\n");
        return 1;
    }
    // Producer: write message into pipe
    const char *message = "Hello, message queue simulation!";
    if (!WriteFile(hWrite, message, strlen(message) + 1, &bytesWritten, NULL)) {
        printf("Error writing to pipe\n");
        return 1;
    }
    printf("Producer: Data sent to pipe: %s\n", message);
```

```
    // Consumer: read message from pipe
    if (!ReadFile(hRead, buffer, sizeof(buffer), &bytesRead, NULL)) {
        printf("Error reading from pipe\n");
        return 1;
    }
    printf("Consumer: Data received from pipe: %s\n", buffer);
    // Close pipe handles
    CloseHandle(hRead);
    CloseHandle(hWrite);
    return 0;
```

OUTPUT:



QUESTION:11

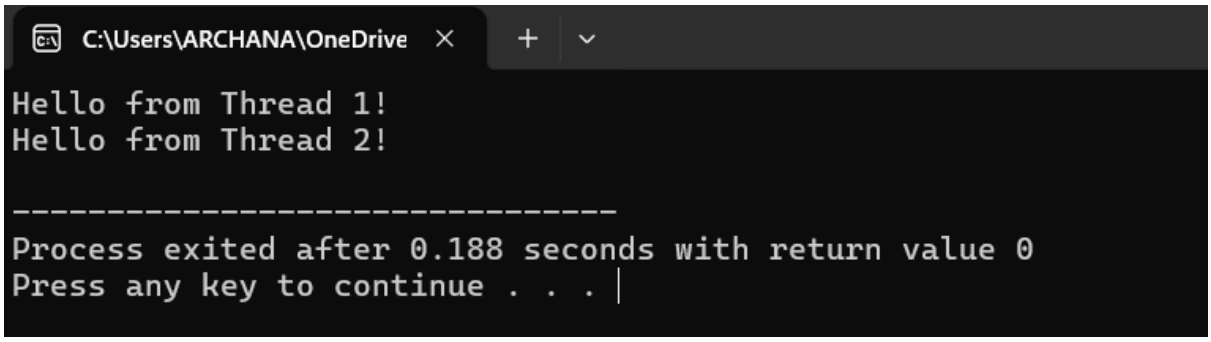
Illustrate the concept of multithreading using a C program.

PROGRAM:

```
#include <stdio.h>
#include <windows.h
// Thread function
DWORD WINAPI threadFunction(LPVOID arg) {
    char* message = (char*)arg;
    printf("%s\n", message);
    return 0;
}
int main() {
```

```
    HANDLE thread1, thread2;
    DWORD threadID1, threadID2;
    char* message1 = "Hello from Thread 1!";
    char* message2 = "Hello from Thread 2!"
    // Create threads
    thread1 = CreateThread(NULL, 0, threadFunction, (LPVOID)message1, 0, &threadID1);
    thread2 = CreateThread(NULL, 0, threadFunction, (LPVOID)message2, 0, &threadID2)
    // Wait for threads to complete
    WaitForSingleObject(thread1, INFINITE);
    WaitForSingleObject(thread2, INFINITE);
    // Close thread handles
    CloseHandle(thread1);
    CloseHandle(thread2);
    return 0;
}
```

OUTPUT:



QUESTION:12

Design a C program to simulate the concept of Dining-Philosophers problem.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
```

```
#define NUM_PHILOSOPHERS 5
#define EAT_COUNT 3   // each philosopher will eat 3 times
pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
void* philosopherLifeCycle(void* arg) {
    int id = *((int*)arg);
    int left_chopstick = id;
    int right_chopstick = (id + 1) % NUM_PHILOSOPHERS;
    for (int i = 0; i < EAT_COUNT; i++) {
        // Think
        printf("Philosopher %d is thinking...\n", id);
        sleep(1);
        // Pick up chopsticks
        pthread_mutex_lock(&chopsticks[left_chopstick]);
        pthread_mutex_lock(&chopsticks[right_chopstick]);
        // Eat
        printf("Philosopher %d is eating... (meal %d)\n", id, i + 1);
        sleep(rand() % 2 + 1); // Eating time
        // Put down chopsticks
        pthread_mutex_unlock(&chopsticks[left_chopstick]);
        pthread_mutex_unlock(&chopsticks[right_chopstick]);
    }
    printf("Philosopher %d has finished eating and leaves.\n", id);
    return NULL;
}
int main() {
    pthread_t philosophers[NUM_PHILOSOPHERS];
    int philosopher_ids[NUM_PHILOSOPHERS];

    // Initialize mutex locks
    for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
```

```
        pthread_mutex_init(&chopsticks[i], NULL);
    }
    // Create philosopher threads
    for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
        philosopher_ids[i] = i;
        pthread_create(&philosophers[i], NULL, philosopherLifeCycle,
                       (void*)&philosopher_ids[i]);
    }
    // Wait for threads to finish
    for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
        pthread_join(philosophers[i], NULL);
    }
    // Destroy mutex locks
    for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
        pthread_mutex_destroy(&chopsticks[i]);
    }
    printf("\nAll philosophers have finished eating. Simulation ends.\n");
    return 0;
}
```

OUTPUT:

